

Cybersecurity · GS 2026.2

Auditoria de privacidade do Aquenda

Mapeamento de PII, taint analysis do app, tradução das falhas em risco para a pessoa e o painel Risco Cidadão.

Integrantes	Giovanne Charelli Zaniboni Silva · RM 556223 Gustavo Oliveira de Moura · RM 555827 Lynn Bueno Rosa · RM 551102
Repositório	github.com/Lynnbrosa/GS-CYBER-AQUENDA
Resultado	Risco Cidadão do protótipo: **29** → **0** depois das correções · exemplo com falhas: **67**
Data	06/11/2026

1 Em uma página

No Aquenda, vazar um dado não é só multa da LGPD. A agenda de hormônio, o nome de registro ou uma conversa sobre HIV expõem a pessoa a discriminação e violência. Este módulo procura esse risco no próprio Aquenda (app, API C#, FastAPI, bancos e logs), mostra o caminho de cada vazamento e explica cada falha pelo que ela significa para quem usa o app.

29
Risco Cidadão do protótipo na primeira auditoria (moderado)

0
depois das correções na API C# (baixo, nos três componentes)

9 de 9
vazamentos plantados que a taint analysis encontrou, sem falso positivo

Missão 01
Mapeamento de PII

15 regras com validação de verdade: CPF, CNS, saúde, nome de registro, senha, tokens. `aquenda_cyber/pii/`

Missão 02
Taint analysis

Segue o dado sensível da fonte até cada saída insegura, entre funções, e barra em tempo de execução. `taint_ts/` e `guarda/`

Missão 03
Tradução via IA

O Claude transforma a falha técnica em alerta de risco para a pessoa, recebendo só a evidência mascarada. `ia/`

2 Mapeamento de PII

Cada regra diz o tipo do dado, a categoria da LGPD e por que achou. O texto passa por normalização (NFKC e remoção de caracteres invisíveis) antes da busca, e o dado sai do motor já mascarado.

Dado	Categoria LGPD	Como o motor confirma
CPF	Dado pessoal	Dígitos verificadores do CPF conferidos.
Cartão Nacional de Saúde (CNS)	Sensível	Passa na verificação do Cartão SUS (módulo 11). Liga a pessoa ao prontuário.
RG	Dado pessoal	Só conta com palavra de contexto por perto (RG, identidade).
Cartão de crédito	Credencial	Passa no algoritmo de Luhn.
Telefone	Dado pessoal	DDD que existe no Brasil e formato de fixo ou celular.
E-mail	Dado pessoal	Formato de endereço com domínio.
Endereço residencial	Dado pessoal	Só conta com sinal de moradia: “moro”, “minha casa”, “resido”.
CEP residencial	Dado pessoal	Só conta com a palavra CEP por perto.
Localização precisa	Dado pessoal	Latitude e longitude com 5 casas ou mais (cerca de 1 metro).
Data de nascimento	Dado pessoal	Data ao lado de “nascimento” ou “nasci em”.
Senha ou PIN	Credencial	Valor depois de “senha” ou “PIN”; placeholders como changeme e <senha> não contam.
Token ou chave de acesso	Credencial	Padrões de chave (sk-ant, AKIA, JWT, Bearer, chave=valor) e corte de entropia.
Dado de saúde	Sensível	Hormônio ou remédio com dose, condição com sinal de que é sobre alguém, código CID.
Nome de registro (nome morto)	Sensível	Nome escrito ao lado de “nome de registro” ou “nome morto”.
Identidade de gênero ou orientação	Sensível	Dita em primeira pessoa: “sou travesti”, “me assumi bi”.

Exemplo

Entrada: “Oi, sou a Bia, CPF 529.982.247-25, tomo estradiol 2mg desde março e meu nome de registro é João. Meu cartão do SUS: 123456789010000. Senha: Kiwi#2026”

Achado	Como aparece no relatório
CPF	`•••.•••.•••-25`
Dado de saúde	`•••••••••`
Nome de registro (nome morto)	`J•••`
Cartão Nacional de Saúde (CNS)	`••••••••••••00`
Senha ou PIN	`•••••••••`

Texto redigido: Oi, sou a Bia, CPF [CPF], tomo [DADO DE SAÚDE] 2mg desde março e meu nome de registro é [NOME DE REGISTRO (NOME MORTO)]. Meu cartão do SUS: [CARTÃO NACIONAL DE SAÚDE (CNS)]. Senha: [SENHA OU PIN]

Contra falso positivo

- “Preparar o paciente” e “Atender pacientes do CAPS” não viram dado de saúde: precisa de dose, de “deu positivo” ou de “meu resultado”.
- Placeholders (`changeme` , `TOKEN-DE-TESTE` , `<senha>`) e o telefone público de um CRAS não contam.
- 38 testes, alguns por propriedade (Hypothesis). O Hypothesis achou um limite real do CPF, que ficou documentado em teste.

3 Taint analysis do app

O analisador lê o app TypeScript com o compilador e o TypeChecker. A política (`politica.json`) diz o que é fonte de dado sensível, o que é saída insegura, o que limpa o dado e quais fluxos são permitidos.

- **Sensível a campo:** `lembrete.id` pode sair; `lembrete.anotacao` não.
- **Entre funções:** cada função ganha um resumo (qual parâmetro chega a qual saída), calculado até estabilizar. Pega o vazamento que passa por um utilitário genérico sem marcar tudo que o chama.
- **React:** segue `useState` , `useMemo` , callbacks de `.map` e `.then` .
- **Sanitizadores:** hash, coordenada arredondada e o texto discreto da notificação. Um fluxo permitido só passa se o dado foi limpo no caminho.

Alvo	Arquivos	Funções	Violações	Fluxos permitidos
App Aquenda	52	259	**0**	18
Exemplo com vazamentos plantados	5	21	**9 de 9**	1

A trilha de um vazamento entre funções

A agenda lida do SQLite passa por `resumoDoDia()` e `enviar()` até chegar ao `fetch` , em `src/util.ts:6` :

fonte: lembretes de hormônio lidos do SQLite → entra em resumoDoDia() → parâmetro → map() → join() → retorna → = texto → entra em enviar() → parâmetro → stringify() → **sai por fetch()**

Os 9 vazamentos encontrados no exemplo

Dado	Sai por	Onde
Agenda de hormônio	rede (fetch)	`src/app/agenda/formulario.tsx:8`
Localização precisa	rede (fetch)	`src/posicao.ts:21`
Localização precisa	rede (fetch)	`src/posicao.ts:8`
Agenda de hormônio	rede (fetch)	`src/util.ts:6`
Agenda de hormônio	rede (fetch)	`src/vazamentos.ts:10`
Consulta médica	link externo	`src/vazamentos.ts:22`
Agenda de hormônio	log	`src/vazamentos.ts:28`
Agenda de hormônio	AsyncStorage sem cifra	`src/vazamentos.ts:34`
PIN do app	log	`src/vazamentos.ts:39`

O zero do app real é resultado da análise e não da falta dela: um teste de mutação copia o app, insere um `fetch` com a agenda e confere que a violação aparece.

4 Bloqueio em tempo de execução

A análise estática acha o problema no código. As travas seguram o que escapar.

Trava	Onde	O que faz
GuardaDeSaida	API (ASGI)	Varre o JSON de cada resposta. Mascara o dado sensível ou barra a resposta inteira com 502. O cabeçalho `x-aquenda-pii` conta os achados.
FiltroPII	logging	O log grava só a versão mascarada: `Login de [CPF] com Bearer [TOKEN OU CHAVE DE ACESSO]`.
saida_segura()	código Python	Confere o valor antes de mandar para um destino que não deve receber dado sensível.
saidaSegura.ts	app, dentro de `pedirJson`	Pedido com CPF, CNS, cartão, saúde, nome de registro ou senha nem sai do celular: vira `ErroApi('bloqueado')`.

A mesma rota, com e sem a guarda

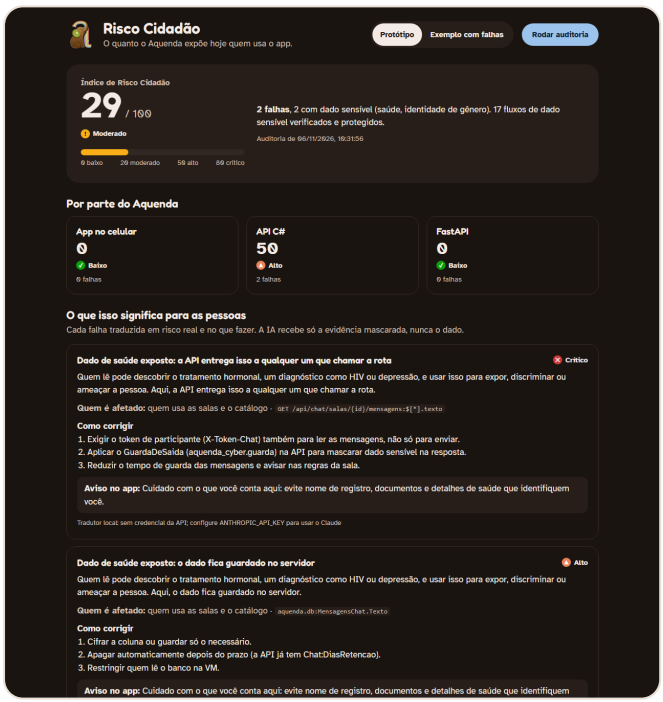
Modo	HTTP	O que sai
sem guarda	200	`"Comecei estradiol 2mg semana passada…" · "Meu CPF é 529.982.247-25…"`
mascarar	200	`"Comecei [DADO DE SAÚDE] 2mg semana passada…" · "Meu CPF é [CPF]…"`
bloquear	502	`"A resposta tinha dado pessoal e não saiu do servidor." tipos: CPF, Dado de saúde`

5 Auditoria do protótipo e o painel Risco Cidadão

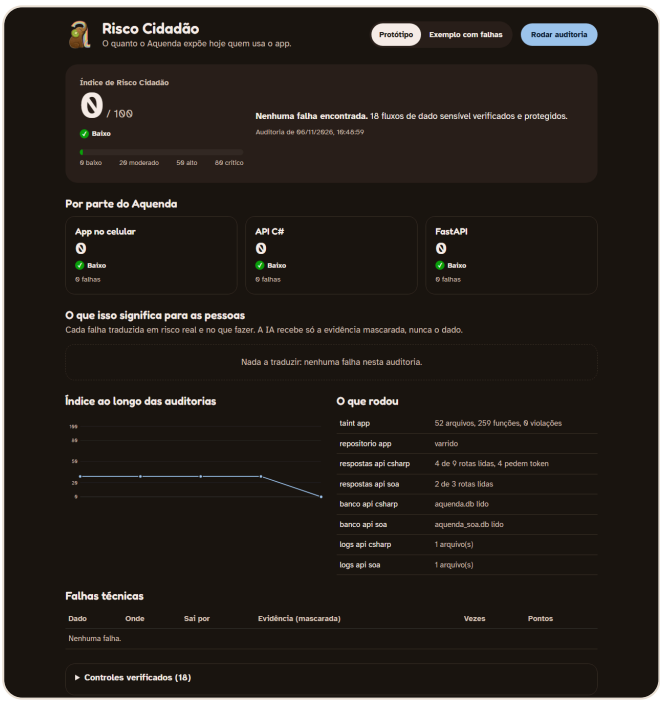
A auditoria junta tudo num modelo único de falha: taint do app, respostas das duas APIs, os dois bancos, os logs e o repositório.

```
pontos = impacto da categoria × peso da exposição × confiança × (1 + log10(ocorrências))
índice  = 100 × (1 − e^(−pontos / escala))
faixas: 0–19 baixo · 20–49 moderado · 50–79 alto · 80–100 crítico
```

Falha encontrada	Onde	Correção
Dado de saúde nas mensagens das salas, lido sem autenticação	`GET /api/chat/salas/{id}/mensagens`	Ler e entrar no tempo real (SignalR) passaram a pedir o token de participante. Sem token: 401.
O mesmo texto guardado em claro	`MensagensChat.Texto`	Texto cifrado com AES-256-GCM antes de ir para o banco. A chave fica fora do banco; mensagens antigas cifradas na inicialização.



Antes: 29, moderado. API C# em alto.



Depois: 0, baixo. O histórico mostra a queda.

O protótipo agora tem **0/100** e 18 fluxos de dado sensível verificados e protegidos. O exemplo com falhas plantadas dá **67/100 (alto)**: app 80 (crítico), API C# 46 pelo log vulnerável.

6 Tradução via IA

Cada falha vira um alerta para pessoas: o risco para quem teve o dado exposto, quem é afetado, a gravidade, como corrigir e um aviso que pode aparecer no app. O modelo é o `claude-opus-5-5`, com saída estruturada (Pydantic).

- **O modelo nunca vê o dado.** Recebe o tipo, a categoria da LGPD, por onde sai, quantas vezes apareceu e a evidência mascarada (•••.•••.•••-25). Um teste confere que o CPF original não está no que vai para a API.
- **Tom de quem cuida:** linguagem simples, sem culpar a pessoa e sem jargão.
- **Sempre tem resposta:** sem chave, sem rede ou com recusa do modelo, o tradutor local gera o alerta com as mesmas regras de gravidade. O cartão diz quem gerou o texto.
- **Custo baixo:** esforço baixo, só as 8 falhas mais graves por auditoria e cache das respostas.

Alertas gerados no exemplo com falhas

Consulta médica: o dado vai para outro app ou site

✕ Crítica

Revela onde e quando a pessoa vai a uma consulta (por exemplo no ambulatório trans), o que expõe a transição e permite encontrá-la. Aqui, o dado vai para outro app ou site.

Quem é afetado: quem usa o app no celular

Como corrigir

1. Remover a chamada que leva o dado para fora do celular.
2. Se for preciso enviar, mandar só um valor derivado (contagem, hash, data) e registrar o fluxo na política do taint.
3. Rodar o taint analysis de novo (npm run taint) para confirmar zero violações.

Aviso no app: Cuidado com o que você conta aqui: evite nome de registro, documentos e detalhes de saúde que identifiquem você.

Dado de saúde: o dado fica gravado em log, que muita gente da equipe e de...

▲ Alta

Quem lê pode descobrir o tratamento hormonal, um diagnóstico como HIV ou depressão, e usar isso para expor, discriminar ou ameaçar a pessoa. Aqui, o dado fica gravado em log, que muita gente da equipe e de ferramentas lê.

Quem é afetado: quem usa as salas e o catálogo

Como corrigir

1. Instalar o FiltroPII (aquenda_cyber.guarda.instalar_filtro) nos loggers da API.
2. Rodar a auditoria de novo para confirmar.

Aviso no app: Cuidado com o que você conta aqui: evite nome de registro, documentos e detalhes de saúde que identifiquem você.

Token ou chave de acesso: o dado fica gravado em log, que muita gente da...

▲ Alta

Quem tiver a chave pode agir em nome do sistema ou da pessoa: ler, apagar ou mandar dados. Aqui, o dado fica gravado em log, que muita gente da equipe e de ferramentas lê.

Quem é afetado: quem usa as salas e o catálogo

Como corrigir

1. Instalar o FiltroPII (aquenda_cyber.guarda.instalar_filtro) nos loggers da API.
2. Rodar a auditoria de novo para confirmar.

Aviso no app: Cuidado com o que você conta aqui: evite nome de registro, documentos e detalhes de saúde que identifiquem você.

7 Como reproduzir

Preparar

```
python -m venv .venv
.venv/Scripts/pip install -r requirements.txt
npm install
```

Rodar

```
python -m aquenda_cyber varrer "CPF 529.982.247-25, tomo estradiol 2mg"
python -m aquenda_cyber taint ../GS-MOBILE-AQUENDA
python -m aquenda_cyber taint taint_ts/exemplos/vulneravel
python -m aquenda_cyber auditar --exemplo
python -m aquenda_cyber painel          # http://localhost:8600
python exemplos/demo_guarda.py
python -m pytest                        # 63 testes
```

Para os alertas saírem do Claude, defina `ANTHROPIC_API_KEY` antes de abrir o painel e clique em “Rodar auditoria”.

Evidências no repositório

- `docs/evidencias/painel-prototipo-antes.png` e `painel-prototipo.png`: o painel antes e depois das correções.
- `painel-exemplo.png`: o painel do exemplo com falhas.
- `taint-app-aquenda.txt`, `taint-exemplo-vulneravel.txt` e `.json`: a taint analysis com a trilha de cada violação.
- `pii-varrer-texto.txt`, `pii-varrer-log.txt`, `guarda-demo.txt` e `testes.txt`.